

ProductionManagementAI — Production Order Schema — Database Design Document (テーブル定義書)

001_DB — requirements REQ-010–REQ-018 (`work-items/WI-002/brief.md`), basic design 001_BD (`docs/en/010_basic-design/001/001_BD_production-order-create-edit.md`), implements 001_DD and 001_DD-API (`docs/en/020_detailed-design/`).

Extended by 003_DB (`003_DB_completion-tracking-and-dashboard-queries.md` , WI-004): `production_orders` gains `completed_at_utc` and two check constraints. The table definition below is 001_DB's and is not rewritten; read 003_DB alongside it.

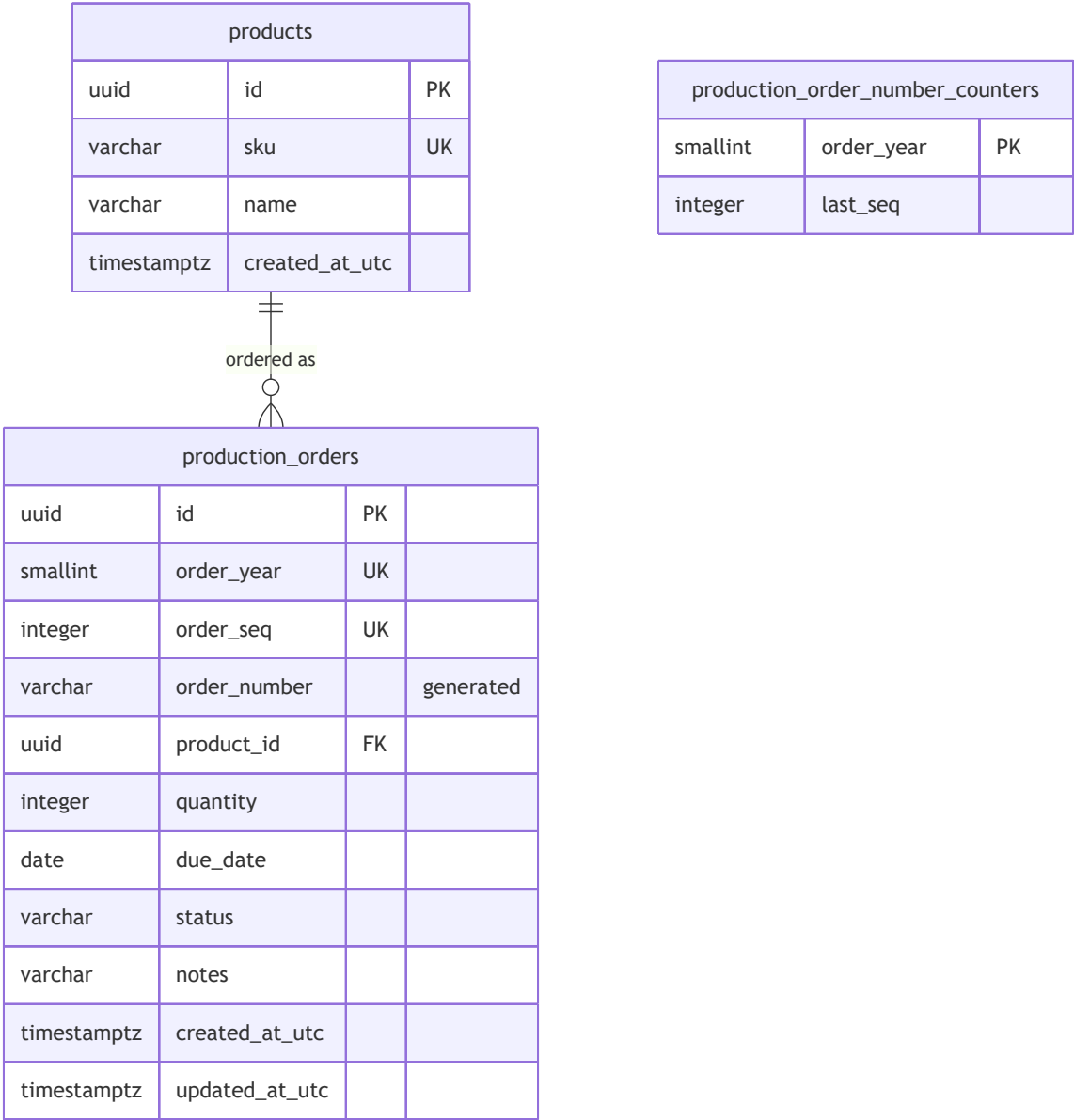
Revised by WI-005 (2026-09-23): the 30 seeded product names are Japanese automobile parts (seed table below), applied to existing databases by the data-only migration `LocalizeDemoDataToJapanese` (see Migration impact). No table, column or constraint changes.

Physical naming follows 000_DB: `snake_case` tables and columns (`EFCore.NamingConventions` , `UseSnakeCaseNamingConvention()` in `DependencyInjection.cs`), `uuid` surrogate keys defaulting to `gen_random_uuid()` , `timestampz` UTC timestamps suffixed `_utc` , and constraint names in the `pk_ / fk_ / ix_ / ck_` style EF Core generates.

Table list

Table	Physical name	Purpose
Products	<code>products</code>	Minimal product reference data, seeded, no catalog UI (DEC-005)
Production orders	<code>production_orders</code>	One row per production order created on SCR-001
Production order number counters	<code>production_order_number_counters</code>	Last issued order-number sequence per year, used to generate <code>P0-YYYY-NNNNN</code> (DEC-012, DEC-013)

ER diagram and relationships



production_order_number_counters has no foreign key: it is a counter keyed by the same year value as production_orders.order_year , not a parent row.

Table	Related table	Relationship (1:1 / 1:N / N:N)	FK column
products	production_orders	1:N (every order references exactly one product)	production_orders.product_id

Table definitions

products

Item name	Physical name	Data type	Length	NOT NULL	Default	PK/FK	Notes
Id	id	uuid	—	yes	gen_random_uuid()	PK	Seed rows use fixed UUIDs so seeding is reproducible
SKU	sku	varchar	50	yes	none	—	Unique business code; shown as "{SKU} — {name}" (001_BD M-03), e.g. P-1004 – ドライブシャフト
Name	name	varchar	200	yes	none	—	Japanese since WI-005; 200 characters is ample for Japanese part names
CreatedAtUtc	created_at_utc	timestampz	—	yes	now()	—	

Seed data: 30 sample products for the demo (DEC-019), inserted by the migration via EF Core HasData with fixed UUIDs. These are illustrative values, not real master data. Since WI-005 the demo is an automobile-parts manufacturer and the names are Japanese (WI-005 DEC-002); SKUs and UUIDs are unchanged, so every order and figure that refers to a product still does.

sku	name	English gloss	Name before WI-005
P-1001	ブレーキキャリパー	brake caliper	Steel bracket
P-1002	ブレーキディスクローター	brake disc rotor	Aluminium housing
P-1003	ブレーキパッド	brake pad	Control panel assembly
P-1004	ドライブシャフト	drive shaft	Drive shaft
P-1005	等速ジョイント	constant-velocity joint	Hydraulic valve
P-1006	トランスミッションケース	transmission case	Gearbox assembly
P-1007	ハブベアリング	hub bearing	Bearing housing

sku	name	English gloss	Name before WI-005
P-1008	ウォーターポンプ	water pump	Pump impeller
P-1009	エンジンマウント	engine mount	Motor mount plate
P-1010	ラジエーター	radiator	Conveyor roller
P-1011	タイミングギア	timing gear	Spur gear 40T
P-1012	クラッチディスク	clutch disc	Coupling flange
P-1013	ショックアブソーバー	shock absorber	Pneumatic cylinder
P-1014	コイルスプリング	coil spring	Sensor bracket
P-1015	エンジンワイヤーハーネス	engine wire harness	Cable harness A
P-1016	ボディワイヤーハーネス	body wire harness	Cable harness B
P-1017	ヘッドランプユニット	headlamp unit	Terminal block unit
P-1018	リレーボックス	relay box	Relay module
P-1019	オルタネーター	alternator	Power supply unit
P-1020	ECU ケース	ECU case	PLC enclosure
P-1021	インタークーラー	intercooler	Heat sink
P-1022	電動ファン	electric cooling fan	Cooling fan assembly
P-1023	オイルフィルター	oil filter	Filter housing
P-1024	スロットルボディ	throttle body	Valve body
P-1025	ピストン	piston	Piston rod
P-1026	コネクティングロッド	connecting rod	Spring assembly
P-1027	サブフレーム	subframe	Welded frame
P-1028	ドアパネル	door panel	Guard panel
P-1029	ドアヒンジ	door hinge	Hinge set
P-1030	ボルト・ナットキット	bolt and nut kit	Fastener kit

production_orders

Item name	Physical name	Data type	Length	NOT NULL	Default	PK/FK	Notes
Id	id	uuid	—	yes	gen_random_uuid()	PK	Used in the edit route <code>/production-orders/{id}</code>
OrderYear	order_year	smallint	—	yes	none	—	Year of creation in the plant timezone, <code>Asia/Tokyo</code> (DEC-011, DEC-017); set by the application, never changed
OrderSeq	order_seq	integer	—	yes	none	—	1–99999 within <code>order_year</code> ; issued from <code>production_order_number_counters</code> (DEC-013); never changed
OrderNumber	order_number	varchar	13	yes	generated	—	<code>GENERATED ALWAYS AS ('PO-' order_year::text '-' lpad(order_seq::text, 5, '0')) STORED</code> , e.g. <code>PO-2026-00001</code> (DEC-012). Read-only to the application
ProductId	product_id	uuid	—	yes	none	FK → <code>products.id</code>	Changeable only while <code>status = 'Draft'</code> (REQ-018, application-enforced)
Quantity	quantity	integer	—	yes	none	—	<code>> 0</code> (REQ-013); changeable only while <code>Draft</code>
DueDate	due_date	date	—	yes	none	—	" <code>≥ today</code> " is application-enforced only (see Constraints)
Status	status	varchar	20	yes	'Draft'	—	<code>Draft</code> / <code>InProgress</code> / <code>Completed</code> / <code>Cancelled</code> (DEC-015); transitions application-enforced (REQ-017)
Notes	notes	varchar	500	no	none	—	NULL when empty (REQ-016)
CreatedAtUtc	created_at_utc	timestamptz	—	yes	<code>now()</code>	—	
UpdatedAtUtc	updated_at_utc	timestamptz	—	yes	<code>now()</code>	—	Set by the application on every successful update
(row version)	xmin (system column)	xid	—	—	—	—	Not a declared column. PostgreSQL's row-version system column, mapped by EF Core as the optimistic-concurrency token (DEC-010, DEC-014)

production_order_number_counters

Item name	Physical name	Data type	Length	NOT NULL	Default	PK/FK	Notes
OrderYear	order_year	smallint	—	yes	none	PK	One row per year that has orders
LastSeq	last_seq	integer	—	yes	none	—	Last <code>order_seq</code> issued for that year; <code>1–99999</code>

Order-number issuance (DEC-013), in the same transaction as the order insert:

```
INSERT INTO production_order_number_counters (order_year, last_seq)
VALUES (@year, 1)
ON CONFLICT (order_year) DO UPDATE SET last_seq = production_order_number_counters.last_seq + 1
RETURNING last_seq;
```

The upsert takes a row lock on that year's counter, so concurrent creates in the same year are serialized for the rest of the transaction and always get distinct numbers. If the insert fails and the transaction rolls back, the counter increment rolls back too, so numbers are not skipped. The statement is issued through the ORM's parameterized raw-SQL API ([ai/rules/database.md](#)).

Index definitions

Index name	Table	Column(s)	Type	Rationale (access pattern)
pk_products	products	id	unique btree (PK)	Product lookup by ID when validating <code>product_id</code> (V-01)
ix_products_sku	products	sku	unique btree	Enforces unique SKU; the product dropdown is ordered by SKU (FN-004)
pk_production_orders	production_orders	id	unique btree (PK)	Load/update by ID (FN-002, FN-003)
ix_production_orders_order_year_order_seq	production_orders	order_year, order_seq	unique btree	Guarantees order-number uniqueness (DEC-006); <code>order_number</code> is derived from these two columns, so it is unique too
ix_production_orders_product_id	production_orders	product_id	btree	Added during implementation (DEC-029): EF Core's foreign-key convention always creates it. Serves the <code>ON DELETE RESTRICT</code> check and Screen B's expected product filter
pk_production_order_number_counters	production_order_number_counters	order_year	unique btree (PK)	Counter upsert by year

Deliberately not added:

- ~~ `production_orders.product_id` ~~: originally left out, but added during implementation (DEC-029). See the index table.

- **production_orders.order_number** : uniqueness is already covered by the (order_year, order_seq) index, and Screen A never looks orders up by number. Screen B's search design decided this: 002_DB adds a pg_trgm GIN index for its case-insensitive fragment search (WI-003 DEC-010), not a btree.

Constraints

Constraint	Type (PK/FK/UNIQUE/CHECK)	Table.column(s)	References	Rule
pk_products	PK	products.id	—	—
ix_products_sku	UNIQUE	products.sku	—	—
pk_production_orders	PK	production_orders.id	—	—
fk_production_orders_products_product_id	FK	production_orders.product_id	products.id	ON DELETE RESTRICT — a product with orders can't be deleted (REQ-015)
ix_production_orders_order_year_order_seq	UNIQUE	production_orders.(order_year, order_seq)	—	—
ck_production_orders_quantity_positive	CHECK	production_orders.quantity	—	quantity > 0 (REQ-013)
ck_production_orders_status	CHECK	production_orders.status	—	status IN ('Draft', 'InProgress', 'Completed', 'Cancelled') (REQ-017)
ck_production_orders_order_seq_range	CHECK	production_orders.order_seq	—	order_seq BETWEEN 1 AND 99999 — keeps NNNNN at five digits (DEC-013)
ck_production_orders_order_year_range	CHECK	production_orders.order_year	—	order_year BETWEEN 2000 AND 9999 — keeps YYYY at four digits
pk_production_order_number_counters	PK	production_order_number_counters.order_year	—	—
ck_production_order_number_counters_last_seq_range	CHECK	production_order_number_counters.last_seq	—	last_seq BETWEEN 1 AND 99999 . If a year's 100,000th order is attempted, the counter upsert fails, the whole create is rolled back and the API returns a server error (DEC-013)

Rules deliberately **not** enforced in the database, and why:

Rule	Why application-only
Due date ≥ today (REQ-014, DEC-009, DEC-011)	"Today" moves, and depends on the plant timezone. A CHECK would make valid historical rows fail on restore (<code>pg_restore</code>) or re-validation, and it can't express "only when changed"
Allowed status transitions (REQ-017)	Transitions depend on the previous value. A trigger could enforce them, but the Domain layer is the single source of the rule (0001_ADR); the CHECK above still blocks unknown values
Product/quantity locked after Draft (REQ-018, DEC-007)	Also depends on the previous row state; enforced in the Domain/Application layer and covered by tests

Transactions and concurrency

Operation	Transaction scope	Concurrency control
Create (FN-001)	Single transaction: counter upsert → insert <code>production_orders</code>	Counter row lock serializes same-year creates; unique <code>(order_year, order_seq)</code> as a backstop
Update (FN-002)	Single <code>UPDATE ... WHERE id = @id AND xmin = @version</code> (EF Core)	Optimistic: 0 rows affected → the order was changed since it was loaded → rejected (V-08, DEC-010). The client sends back the <code>version</code> it loaded
Read (FN-003, FN-004)	No explicit transaction	—

Application database privileges

Least privilege this feature needs (`ai/rules/database.md`):

Table	Privileges
<code>products</code>	SELECT
<code>production_orders</code>	SELECT, INSERT, UPDATE (no DELETE — orders are never deleted)
<code>production_order_number_counters</code>	SELECT, INSERT, UPDATE

Login split (DEC-016, decided: split now):

Login	Used by	Privileges
Owner (<code>POSTGRES_USER</code> , existing)	<code>dotnet ef database update</code> only	Owns the schema; runs DDL
Runtime (new, proposed name <code>pmai_app</code>)	The backend at runtime (<code>ConnectionStrings__DefaultConnection</code>)	<code>CONNECT</code> on the database, <code>USAGE</code> on schema <code>public</code> , the table grants above, plus what ASP.NET Core Identity and <code>IdentitySeeder</code> need on the 000_DB tables: <code>SELECT/INSERT/UPDATE</code> on <code>users</code> , <code>roles</code> , <code>user_roles</code> , <code>user_claims</code> , <code>role_claims</code> , <code>user_logins</code> , <code>user_tokens</code> , <code>DELETE</code> on <code>user_roles</code> / <code>user_claims</code> / <code>user_logins</code> / <code>user_tokens</code> (Identity removes these rows), and <code>USAGE</code> , <code>SELECT</code> on the identity sequences of <code>user_claims.id</code> / <code>role_claims.id</code> . No DDL, no <code>TRUNCATE</code>

Implemented (plan revision 2): the `AddProductionOrders` migration, which runs as the owner, creates `pmai_app` as `NOLOGIN` if it's missing and applies these grants. `deploy/db/init/10-app-login.sh` , baked into the database image, gives it `LOGIN PASSWORD` from `PMAI_APP_DB_PASSWORD` on a fresh volume, so no password is in the repository. No `ALTER DEFAULT PRIVILEGES` blanket grant is used: any later migration that adds a table must extend the script with that table's explicit grants. The exact grant list is verified by the integration tests, which run the app as the runtime login, and by `RuntimeLogin_CannotDeleteOrdersOrRunDdl` .

API / DD mapping

DD field names are 001_BD \$3 variable names; API field names are confirmed in 001_DD-API. 001_DD-API adds two computed response-only fields (`allowedNextStatuses` , `isProductQuantityEditable`) that have no column.

Table.column	DD field	API field
<code>production_orders.id</code>	(route <code>id</code>)	<code>id</code>
<code>production_orders.order_number</code>	<code>orderNumber</code>	<code>orderNumber</code> (response only)
<code>production_orders.product_id</code>	<code>productId</code>	<code>productId</code>
<code>production_orders.quantity</code>	<code>quantity</code>	<code>quantity</code>
<code>production_orders.due_date</code>	<code>dueDate</code>	<code>dueDate</code> (ISO <code>YYYY-MM-DD</code>)
<code>production_orders.status</code>	<code>status</code>	<code>status</code> (<code>Draft</code> <code>InProgress</code> <code>Completed</code> <code>Cancelled</code>); request field on update only — create always yields <code>Draft</code>
<code>production_orders.notes</code>	<code>notes</code>	<code>notes</code> (<code>null</code> when empty)
<code>production_orders.created_at_utc</code>	<code>createdAt</code>	<code>createdAt</code> (response only, ISO 8601 UTC)

Table.column	DD field	API field
production_orders.updated_at_utc	updatedAt	updatedAt (response only, ISO 8601 UTC)
production_orders.xmin	(hidden)	version (in the response; required in update requests)
production_orders.order_year , order_seq	—	not exposed (internal to order-number generation)
products.id	productId options	GET /api/products → id
products.sku , products.name	Product option label (M-03)	GET /api/products → sku , name

Migration impact and recovery limits

- **Migration type:** additive — one new EF Core migration (proposed name `AddProductionOrders`) creates three new tables plus the product seed rows. No existing table is changed.
- **Data recovery limit:** there are no existing rows to lose when applying it. Rolling it back after use permanently deletes every production order and counter row. Take a `pg_dump` of these tables first if their data matters.
- **Rollback plan:** `dotnet ef database update InitialIdentitySchema --project src/backend/ProductionManagementAI.Infrastructure --startup-project src/backend/ProductionManagementAI.Api` drops the three tables. Nothing outside them references them.
- **WI-005 data migration `LocalizeDemoDataToJapanese`:** data only, no DDL. It renames the 30 seeded products by their fixed UUIDs (EF `UpdateData`, from `ProductSeed`), and rewrites the notes of WI-003's seeded orders from `Demo order N` to `デモ用の製造指示 N`, matched by the seed id prefix `0197e4a0-0000-7000-8001-` and by the exact old text, so orders users created, and seeded notes a user edited, are never touched (WI-005 DEC-008). `Down` restores both. It runs as the owner like every migration and needs no new privilege; an existing Compose volume needs no wipe. A user who renamed a seeded product in the database directly would have that name overwritten, which is acceptable because there is no product UI.
- **Operational note:** the generated `order_number` column is defined with EF Core `HasComputedColumnSql(..., stored: true)`. The expression uses explicit `::text` casts because PostgreSQL requires generation expressions to be immutable.

Open decisions

Decision	Options	Recommendation	Status
How to generate the per-year order sequence	per-year counter table with upsert; <code>MAX(order_seq)+1</code> with retry; one PostgreSQL sequence per year	Counter table	decided — see <code>work-items/WI-002/decisions.md</code> DEC-013
What happens past 99,999 orders in one year	reject the create; widen to 6 digits	Reject (CHECK constraint)	decided — DEC-013
Optimistic-concurrency token	PostgreSQL <code>xmin</code> ; explicit <code>version</code> column	<code>xmin</code>	decided — DEC-014
Status storage	<code>varchar</code> + CHECK; PostgreSQL enum type; smallint code	<code>varchar</code> + CHECK	decided — DEC-015
Split migration-owner and restricted runtime DB roles	split now; keep the single owner role for the local demo	Split before any shared/non-local deployment	decided — split now (DEC-016, user)
Demo product seed size	5; ~15; 30	5	decided — 30 (DEC-019, user)